

CS50P: Week 0 Master Reference

Harvard University | Introduction to Programming with Python • Complete Mechanics & Problem Sets

1. The Terminal, Linux Shell & Workflow Architecture

Programs are built and executed inside a terminal shell. Navigating the file system efficiently and invoking command-line tools is foundational for all automated verification (`check50`) and remote submissions (`submit50`).

Command	Full Form / Context	Operational Behavior & Best Practices
<code>pwd</code>	Print Working Directory	Outputs the absolute path of the current shell location (e.g., <code>/workspaces/cs50p</code>).
<code>ls</code>	List	Lists all items in the current folder. Use <code>ls -l</code> for details or <code>ls -a</code> to show hidden files.
<code>cd <dir></code>	Change Directory	Moves down into the specified child directory (e.g., <code>cd tip</code>).
<code>cd ..</code>	Change Directory Parent	Moves one level upward into the enclosing parent directory.
<code>cd ~</code> or <code>cd</code>	Change Directory Home	Immediately returns to the root home directory (<code>/home/ubuntu</code> or workspace root).
<code>mkdir <dir></code>	Make Directory	Allocates a new directory folder. Typical creation workflow: <code>mkdir tip && cd tip</code> .
<code>code <file.py></code>	VS Code Open	Creates and opens the designated source file directly inside the editor buffer.
<code>python <file.py></code>	Python Interpreter	Directly invokes the Python 3 interpreter to run the given script interactively.
<code>check50 <slug></code>	CS50 Grader	Transfers code to an isolated Harvard sandbox, running automated edge-case assertion suites.
<code>submit50 <slug></code>	CS50 Submit	Stages, commits, and pushes verified assignments to the official gradebook via GitHub.

2. Input and Output Primitives

Every program takes raw input, processes values in memory, and presents structured output:

`input(prompt)`

STANDARD IN

Halts execution, displays prompt text, and collects standard input until newline.

Fundamental Rule: `input()` always returns a string (`str`), even when numerical characters are entered.

`print(*objects, sep, end)`

STANDARD OUT

Outputs comma-separated items converted to text.

- `sep=" "` : Separator injected between values (default: space).
- `end="\n"` : Character appended at completion (default: newline).

3. String Methods & String Immutability

Strings (`str`) in Python are strictly **immutable**. A string instance cannot be altered in memory; calling any string method produces a newly allocated string that must be returned or reassigned.

Method	Syntax & Operational Spec	Primary Use Case / Problem Set Context
<code>.lower()</code>	<code>s.lower()</code>	Transforms all alphabetic characters to lowercase (Solved <i>indoor.py</i>).
<code>.upper()</code>	<code>s.upper()</code>	Converts all alphabetic characters to uppercase for strict normalization.
<code>.strip([chars])</code>	<code>s.strip()</code> or <code>s.strip("\$")</code>	Removes whitespace by default, or specific leading/trailing glyphs (Solved <i>tip.py</i>).
<code>.title()</code>	<code>s.title()</code>	Capitalizes each space-delimited word while forcing all remaining characters to lower.
<code>.replace(old, new)</code>	<code>s.replace(" ", "...")</code>	Replaces all occurrences in $\mathcal{O}(N)$ linear time (Solved <i>playback.py</i> & <i>faces.py</i>).
<code>.split(sep)</code>	<code>s.split(" ")</code>	Splits a string by delimiter, returning a list of constituent substring tokens.

Method Chaining: Because string operations return strings, methods can be chained cleanly: `cleaned = raw_text.strip().strip("$").lower()`

4. Data Types, Casting & Formatting

Python enforces dynamic, strongly typed operations. When arithmetic operations are attempted on strings without prior conversion, Python either performs concatenation/replication or raises runtime `TypeError`.

Numeric Casting & Literals

- `int(x)`: Casts valid digit sequences to unbounded precision integers. Rejects floats written as strings.
- `float(x)`: Converts integer or floating-point formatted strings into IEEE 754 64-bit precision floats.
- `300_000_000`: Visual digit separators clarify large constants without affecting evaluation (used in *einstein.py*).

Precision Formatting (f-strings)

Formatted string literals interpolate expressions seamlessly using curly brackets:

```
tip = 14.4
print(f"Leave ${tip:.2f}")
# Output: Leave $14.40
```

`:.2f` guarantees exact rounding to two decimal places and pads trailing zeros.

The String Replication Trap: Writing `"50" * 2` produces `"5050"` (string replication). You must cast explicitly to numbers: `int("50") * 2` produces `100`.

5. Function Architecture, Scoping & The `main()` Convention

Functions compartmentalize logic, isolate variable namespaces, and eliminate redundant operations. Top-level program flow is structured cleanly using the `def main():` convention.

```
def main():
    meal = dollars_to_float(input("Meal cost: "))
    print(f"Total: ${meal:.2f}")

def dollars_to_float(d):
    return float(d.strip("$"))

main() # Invokes the main entry point after defining all functions
```

Local Scope vs. Global Scope

Variables declared within a function block exist strictly inside that function's **local stack frame**. They cannot be accessed by name in `main()` or any other sibling function without explicit parameter passing.

The Role of

`return`

The `return` keyword passes the result of a function's evaluation back to the caller. Omitting `return` causes a function to execute to completion and yield `None` by default.

6. Indentation: Whitespace as Syntax

Unlike C or Java, which delineate code blocks with curly braces `{}`, Python enforces structure strictly through **uniform whitespace indentation**.

The 4-Space Rule (PEP 8): Every nested statement following a colon (`:`) must be indented by exactly **4 spaces** (or 1 tab). Inconsistent indentation triggers immediate `IndentationError` or semantic bugs.

THE "MISALIGNED RETURN" TRAP (THE `NONE` BUG)

If a `return` statement is un-indented back to Column 0 (the outer margin), Python considers the function closed right before that line:

✘ Broken Architecture (Yields None)

```
def dollars_to_float(d):
    d = float(d.strip("$"))
    return d # Sits outside! Function yields None.
```

Runtime Result: In `main()`, `dollars * percent` crashes with `TypeError: unsupported operand for *: 'NoneType'`.

✔ Correct Block Alignment

```
def dollars_to_float(d):
    d = float(d.strip("$"))
    return d # Indented 4 spaces inside!
```

Runtime Result: The function resolves, returns the computed `float`, and arithmetic in `main()` proceeds smoothly.

7. Week 0 Problem Sets: Technical Post-Mortem

A synthesis of all five submitted and graded problem sets, highlighting the underlying mechanisms and test edge cases.

Problem	Core Objective	Key Mechanics	Critical Takeaways & Pitfalls
1. Indoor Voice <code>indoor.py</code>	Lowercasing speech	<code>input()</code> , <code>.lower()</code>	Direct method chaining onto <code>input()</code> or inside <code>print()</code> avoids unneeded reassignments.
2. Playback Speed <code>playback.py</code>	Pacing words with <code>...</code>	<code>.replace(" ", "...")</code>	String substitution operates in $O(N)$ linear time without mutating the source string.
3. Making Faces <code>faces.py</code>	Emoticon to emoji conversion	<code>convert(text)</code> , chained <code>.replace()</code>	Decoupled architecture: pure helper function handles transformation; <code>main()</code> manages I/O.
4. Einstein <code>einstein.py</code>	Compute $E = mc^2$	<code>int()</code> , Exponentiation <code>**</code>	<code>check50</code> strictly enforces the prompt string: <code>input("m: ")</code> . Unbounded int precision prevents overflow.
5. Tip Calculator <code>tip.py</code>	Clean currency strings and compute tips	<code>.strip("\$")</code> , <code>float()</code> , <code>/ 100</code>	<code>d.strip()</code> without parameters only removes spaces. Return statements must be indented within each function block.

8. Pre-Flight Verification Checklist

- **Exact Prompt Strings:** Always check if `check50` specifies exact prompts (e.g., `input("m: ")`).
- **String Reassignment:** Since strings are immutable, remember to store results: `text = text.replace(...)`.
- **Indentation Consistency:** Verify that every `return` aligns under its function's 4-space block.
- **Type Safety:** Cast inputs before performing arithmetic operations (`int()` for whole numbers, `float()` for decimals).